

Parallel and Distributed Computing Notes

WEEK 1 - Introduction to Parallel and Distributed Computing

Q1. Define "computing" in the context of parallel and distributed systems. Why has its need constantly increased?

- **Parallel and distributed computing** uses **multiple computers** to solve large-scale problems over the Internet.
 - Computing has shifted from centralized single machines to **network-centric, data-intensive** systems.
 - Key drivers: billions of Internet users, demand for **real-time services**, emergence of cloud, IoT, and big data.
 - Systems now require **high-throughput computing (HTC)** rather than just high-performance single-task speed.
 - **Keywords:** scalable, data-intensive, network-centric, distributed, cloud, IoT
-

Q2. Define Centralized Computing. How is it different from other paradigms?

- All processors, memory, and storage are **fully shared and tightly coupled** within **one physical system** under **one integrated OS**.
 - It is the simplest paradigm - no network communication or message passing.
 - Many **data centers and supercomputers** are centralized but used for parallel/distributed applications.
 - **Key distinction:** Unlike distributed systems, there is **no message passing** between separate machines.
 - **Keywords:** tightly coupled, single OS, centralized resources, data center
-

Q3. Define Parallel Computing.

- Processors are either **tightly coupled (shared memory)** or **loosely coupled (distributed memory)**.
 - Inter-processor communication via **shared memory** or **message passing**.
 - Programs running on a parallel computer are called **parallel programs**.
 - The act of writing such programs is **parallel programming**.
 - **Keywords:** tightly/loosely coupled, message passing, shared memory, parallel program
-

Q4. Define Distributed Computing.

- Multiple **autonomous computers**, each with its own **private memory**, communicating over a **network**.
 - Information exchange via **message passing**.
 - A program running on such a system = **distributed program**; writing it = **distributed programming**.
 - **Key distinction from parallel:** nodes are fully autonomous; no shared memory.
 - **Keywords:** autonomous, private memory, message passing, network
-

Q5. Define Cloud Computing.

- Can be either **centralized or distributed**; applies both parallel and distributed computing.
 - Built with **physical or virtualized resources** over large data centers.
 - Also referred to as **utility computing** or **service computing**.
 - Enabled by convergence of: hardware virtualization, utility/grid computing, SOA/Web 2.0, and autonomic computing.
 - **Keywords:** virtualization, utility computing, service computing, data center
-

Q6. What is High Performance Computing (HPC)? How is it measured?

- Measured by **floating-point speed** using the **Linpack benchmark**.
 - Targets **less than 10%** of all computer users (scientists, engineers).
 - Focuses on **solving a single problem as fast as possible** (latency-oriented).
 - Example systems: **MPPs (Massively Parallel Processors)** and clusters.
 - **Keywords:** Linpack benchmark, floating-point, MPP, latency
-

Q7. What is High Throughput Computing (HTC)? How does it differ from HPC?

- Focuses on **high-flux computing** - number of **tasks completed per unit of time**.
 - Main application: **Internet searches and web services**.
 - Performance goal = **throughput** (tasks/second), not raw speed.
 - HTC must address: **cost, energy savings, security, and reliability**.
 - **Keywords:** throughput, flux, tasks per unit time, Internet, web services
-

Q8. What are the "Three Walls" of HPC?

- **Memory Wall:** Memory bandwidth limits single-processor performance; gap between processor speed and memory speed keeps growing.
 - **Power Wall:** Millions of nodes each drawing watts = massive energy and cooling challenges (MWs of dissipation).
 - **Reliability Wall:** With millions of nodes, some will always malfunction; ensuring correct operation is non-trivial.
 - **Keywords:** memory wall, power wall, reliability wall, bandwidth, cooling, failure
-

Q9. What is Koomey's Law?

- States that **energy-efficient computing improves exponentially** - computations per kWh **double every 1.57 years**.
 - Analogous to Moore's Law but focused on **energy efficiency**, not transistor count.
 - Like Moore's Law, Koomey's Law is **expected to eventually end**.
 - Applies at both ends: **supercomputers** and **personal electronics**.
 - **Keywords:** energy efficiency, computations per kWh, 1.57 years, exponential
-

Q10. List and explain the five degrees of parallelism in historical order.

- **BLP (Bit-Level Parallelism):** Widening data paths (8 to 16 to 32 to 64 bit) - approximately 50 years ago.
 - **ILP (Instruction-Level Parallelism):** Pipelining, superscalar, VLIW, multithreading.
 - **DLP (Data-Level Parallelism):** SIMD - same instruction on multiple data elements.
 - **TLP (Task-Level Parallelism):** Multicore processors, CMPs.
 - **JLP (Job-Level Parallelism):** Distributed computing across machines.
 - **Keywords:** BLP, ILP, DLP, TLP, JLP, SIMD, VLIW, multicore, pipelining
-

Q11. What is a Cyber-Physical System (CPS)?

- Result of **interaction between computational processes and the physical world**.
 - Integrates **"cyber"** (heterogeneous, asynchronous) with **"physical"** (concurrent, information-dense) objects.
 - Merges **"3C" technologies: Computation, Communication, Control**.
 - Forms an **intelligent closed feedback loop** between the physical and information world.
 - **Keywords:** 3C, cyber, physical, feedback, computation, communication, control
-

Q12. What is GPU Computing and how does it work?

- GPU has a **many-core architecture** with hundreds of **simple processing cores** organized as multiprocessors.
 - Designed for **parallel execution of floating-point operations concurrently**.
 - **CPU instructs the GPU** to perform massive data processing.
 - Bandwidth must be matched between **on-board main memory** and **on-chip GPU memory**.
 - **Keywords:** many-core, floating-point, parallel, on-chip GPU memory, bandwidth
-

Q13. What is the "Memory Wall Problem"?

- **Processor speed** has grown much faster than **memory bandwidth/capacity**.
 - Results in processors waiting idle for data from memory.
 - DRAM capacity grew from **16 KB (1976)** to **64 GB (2011)**, but still lags behind CPU demands.
 - **Keywords:** memory wall, DRAM, bandwidth, processor-memory gap
-

Q14. What is a Virtual Machine (VM)? Why are VMs useful?

- Provides a **software abstraction** that decouples applications from the underlying hardware.
 - A **Virtual Machine Manager (VMM)** manages multiple VMs on one physical machine.
 - Solves: **underutilized resources, application inflexibility, software manageability, security concerns**.
 - Enables running different OS/software on the same physical hardware.
 - **Keywords:** VMM, decoupling, abstraction, inflexibility, security
-

Q15. What are the four technology areas that enable Cloud Computing?

1. **Hardware virtualization and multi-core chips**
 2. **Utility and grid computing**
 3. **SOA and Web 2.0**
 4. **Autonomic computing and data center automation**
- **Keywords:** virtualization, utility, grid, SOA, autonomic
-

Q16. Define Elapsed Time and CPU Time. Give their formulas.

- **Elapsed Time** = wall-clock time from start to finish; includes I/O waits, other programs.
 - $\text{Elapsed Time} = \text{CPU Time} + \text{Wait Time}$
 - **CPU Time** = actual time CPU spends on a program; excludes I/O waits.
 - $\text{CPU Time} = \text{User CPU Time} + \text{System CPU Time}$
 - $\text{Elapsed Time} = \text{User CPU Time} + \text{System CPU Time} + \text{Wait Time}$
 - **Keywords:** elapsed, CPU, user, system, wait, I/O
-

Q17. What is Amdahl's Law? State its formula.

- Gives the **theoretical maximum speedup** for a **fixed workload** when part of it is enhanced.
 - $\text{Speedup}_{\text{overall}} = 1 / [(1 - \text{Fraction}_{\text{enhanced}}) + \text{Fraction}_{\text{enhanced}} / \text{Speedup}_{\text{enhanced}}]$
 - Shows **diminishing returns** as more processors are added.
 - The **serial (non-parallelizable) portion** limits the maximum achievable speedup.
 - **Keywords:** fixed workload, serial fraction, diminishing returns, theoretical maximum
-

Q18. What is the Generalized Amdahl's Law?

- Extends Amdahl's law to **multiple components**, each with a different speedup.
 - If total runtime = 1 = $f_1 + f_2 + \dots + f_k$, with fraction f_i sped up by factor p_i :
 - $S = 1 / (f_1/p_1 + f_2/p_2 + \dots + f_k/p_k)$
 - If a fraction is **slowed down**, use $s_j \times f_j$ instead of f_j/p_j where $s_j > 1$.
 - **Keywords:** multiple fractions, generalized, slowdown factor
-

Q19. What are the four design objectives for future HPC and HTC systems?

- **Efficiency:** Utilization rate of resources; for HTC = job throughput per watt.
 - **Dependability:** Reliability and self-management from chip to system; QoS assurance under failure.
 - **Adaptation:** Support billions of job requests over massive datasets and virtualized cloud.
 - **Flexibility:** Ability to run well in both HPC (science/engineering) and HTC (business) applications.
 - **Keywords:** efficiency, dependability, adaptation, flexibility, QoS
-

Q20. What is the difference between Linpack Benchmark and HTC performance measurement?

- **Linpack:** measures **floating-point speed** (FLOPS); used for HPC/supercomputers.
 - **HTC:** measures **throughput** - number of tasks/transactions completed **per unit time** (TPS, MIPS).
 - As cloud computing emerged, Linpack became **insufficient** for measuring real-world performance.
 - **Keywords:** Linpack, FLOPS, TPS, MIPS, throughput
-

WEEK 2A - Distributed System Fundamentals

Q21. Give the formal definition of a Distributed System.

- "A distributed system is a **collection of autonomous computing elements** that appears to its users as a **single coherent system**."
 - Two key characteristics:
 1. **Autonomous nodes** (hardware devices or software processes).
 2. **Single coherent system** - users perceive it as one, regardless of physical distribution.
 - **Keywords:** autonomous, coherent, nodes, collaboration, middleware
-

Q22. What is meant by "no global clock" in distributed systems?

- Each **autonomous node** has its own notion of time.
 - There is **no shared global clock** across the system.
 - This leads to fundamental **synchronization and coordination problems**.
 - Nodes cannot simply agree on the order of events without special protocols.
 - **Keywords:** no global clock, synchronization, coordination, autonomous
-

Q23. What is an Overlay Network in distributed systems?

- Each node communicates **only with its neighbors** in the overlay.
- The neighbor set may be **dynamic** or known only **implicitly** (via lookup).

- Two types:
 - **Structured**: well-defined set of neighbors (tree, ring).
 - **Unstructured**: references to **randomly selected** other nodes (mesh).
- Classic example: **Peer-to-Peer (P2P) systems**.
- **Keywords**: overlay, structured, unstructured, P2P, neighbor, dynamic

Q24. What is "Distribution Transparency"? List all types.

- Hiding the fact that processes and resources are **physically distributed**.

Transparency	What it Hides
Access	Differences in data representation and how an object is accessed
Location	Where an object is located
Relocation	Object moved to another location while in use (key in cloud)
Migration	Object may move to another location (mobility)
Replication	Object is replicated
Concurrency	Object may be shared by several independent users
Failure	Failure and recovery of an object

- **Keywords**: transparency, hiding, distribution, abstraction

Q25. What is a "Partial Failure" and why is it a challenge?

- In a distributed system, **only part of the system may fail at any time**.
- **Hiding partial failures** and their recovery is very difficult and often **impossible** to fully achieve.
- This is "the snag" of trying to present a single coherent system.
- **Keywords**: partial failure, hiding, recovery, coherence, inevitable

Q26. What are the three dimensions of Scalability?

1. **Size Scalability**: Number of users and/or processes.
 2. **Geographical Scalability**: Maximum distance between nodes.
 3. **Administrative Scalability**: Number of administrative domains.
- Most systems only handle **size scalability**; geographical and administrative remain challenging.
 - **Keywords**: size, geographical, administrative, scalability

Q27. What causes scalability problems in centralized solutions?

- **Computational capacity**: limited by CPUs.
- **Storage capacity**: limited transfer rate between CPUs and disks.
- **Network bandwidth**: bottleneck between users and the centralized service.
- **Keywords**: CPU bottleneck, storage, network, centralized limit

Q28. What are the problems with Geographical Scalability?

- Cannot simply go from **LAN to WAN**: latency prohibits synchronous client-server interactions.
- **WAN links are inherently unreliable**: streaming video over WAN fails.
- **Lack of multipoint communication**: simple broadcast searches do not work; need separate **naming and directory services**.
- **Keywords**: LAN, WAN, latency, unreliable, naming, directory services

Q29. What are the problems with Administrative Scalability?

- **Conflicting policies** on usage, management, payment, and security across different domains.
- Example: **Computational grids** - sharing expensive resources between organizations with different rules.
- Exception: **P2P systems** (BitTorrent, Skype, Spotify) bypass this because **end users** (not administrators) collaborate.
- **Keywords**: policy conflict, administrative domains, grid, P2P, BitTorrent

Q30. List the 8 "False Assumptions" (Pitfalls) made when developing distributed systems.

1. The network is **reliable**
2. The network is **secure**
3. The network is **homogeneous**
4. The **topology does not change**
5. **Latency is zero**
6. **Bandwidth is infinite**
7. **Transport cost is zero**
8. There is **one administrator**

- **Keywords:** false assumptions, pitfalls, reliable, latency, bandwidth, topology
-

Q31. What is Middleware in a distributed system? What services does it provide?

- Middleware is the "**OS of distributed systems**" - sits between the local OS and applications.
 - Provides **commonly used components and functions** so applications do not implement them separately.
 - Services include:
 - **Inter-application communication** (e.g., RPC)
 - **Security services**
 - **Accounting services**
 - **Masking recovery from failure**
 - **Naming protocols, replication, caching**
 - Main difference from OS: middleware services are offered in a **networked manner**.
 - **Keywords:** middleware, RPC, security, naming, replication, networked
-

Q32. What is a "Remote Procedure Call (RPC)"?

- A common **communication service** provided by middleware.
 - Allows a process to **call a procedure on a remote machine** as if it were local.
 - Abstracts network communication from the developer.
 - **Keywords:** RPC, remote, procedure, middleware, transparency
-

Q33. What is an "Atomic Transaction" in middleware?

- Making use of **multiple distributed services** where the result is "**all or nothing**".
 - Either the entire transaction commits or the entire transaction is rolled back.
 - Used for **consistency** across distributed services.
 - **Keywords:** atomic, all or nothing, transaction, consistency, rollback
-

Q34. What are the four design goals of distributed systems?

1. **Resource Sharing** - cloud storage, P2P streaming, shared mail/web hosting.
 2. **Distribution Transparency** - hiding physical distribution from users.
 3. **Openness** - following standard interfaces and protocols.
 4. **Scalability** - handling growth in users, geography, and administration.
- **Keywords:** resource sharing, transparency, openness, scalability
-

WEEK 2B - Types of Distributed Systems

Q35. What are the three main types of distributed systems?

1. **High Performance Distributed Computing Systems**
 2. **Distributed Information Systems**
 3. **Distributed Systems for Pervasive Computing**
- **Keywords:** HPC, distributed information, pervasive computing
-

Q36. What is Cluster Computing? What are its key properties?

- A group of **high-end systems** connected through a **LAN**.
- **Homogeneous:** same OS, near-identical hardware.
- Has a **single managing node** (master node) and multiple **compute nodes**.
- Example: **Linux-based Beowulf Cluster**.

- **Keywords:** LAN, homogeneous, master node, Beowulf, cluster
-

Q37. What is Grid Computing? How does it differ from Cluster Computing?

Feature	Cluster	Grid
Hardware	Homogeneous	Heterogeneous
Network	LAN	WAN
Administration	Single domain	Multiple organizations
Size	Small	Large
Organization	Master-slave	Virtual organizations

- Grid uses **virtual organizations** for authorization and resource allocation.
 - **Keywords:** heterogeneous, virtual organizations, WAN, multi-domain, authorization
-

Q38. What are the four layers of Cloud Computing architecture?

1. **Hardware Layer:** Processors, routers, power, cooling - customers **never see this**.
2. **Infrastructure Layer:** Virtualization - allocates **virtual storage** and **virtual servers**.
3. **Platform Layer:** Higher-level abstractions (e.g., Amazon S3's **bucket API**).
4. **Application Layer:** Actual applications - office suites, Gmail, YouTube (comparable to OS apps).

- **Keywords:** hardware, infrastructure, platform, application, IaaS, PaaS, SaaS
-

Q39. What is Distributed Shared Memory (DSM)? Why was it abandoned?

- Implements a **shared-memory model on top of a multicomputer**.
 - Uses **virtual-memory techniques**: maps all memory pages into one single virtual address space.
 - If process at CPU A accesses page at CPU B, the OS at A **traps and fetches** it from B.
 - **Problem:** Performance could **never compete** with true multiprocessors; failed to meet programmer expectations.
 - **Keywords:** virtual memory, shared model, trap, multicomputer, abandoned
-

Q40. What is a Virtual Organization in Grid Computing?

- A **grouping of users (and their IDs)** that allows **authorization on resource allocation**.
 - Enables collaboration across different **administrative domains**.
 - Allows grids to share expensive resources across organizations with different policies.
 - **Keywords:** virtual organization, authorization, grid, resource allocation, domain
-

Q41. What are the five layers in Grid Architecture?

1. **Fabric Layer** - interfaces to local resources (query state, lock).
2. **Connectivity Layer** - communication/transaction protocols; authentication.
3. **Resource Layer** - manages a **single resource** (create processes, read data).
4. **Collective Layer** - handles **multiple resources**: discovery, scheduling, replication.
5. **Application Layer** - actual grid applications within a single organization.

- **Keywords:** fabric, connectivity, resource, collective, application, grid layers
-

Q42. Is Cloud Computing always cost-effective? What factors determine this?

- Not necessarily - **outsourcing is only cheaper when monetary benefits exceed extra Internet communication costs**.
 - A migration plan must ensure:
 1. **Policy constraints are met.**
 2. **Additional latencies do not violate delay constraints.**
 3. **All transactions operate correctly; no data loss.**
 - Each component C_i has N_i servers; the plan determines how many n_i should migrate.
 - **Keywords:** outsourcing, migration, latency constraints, policy, cost-effective
-

WEEK 3 - Communications

Q43. Why do communicating processes P and Q need to agree on the "meaning of bits"?

- Even if P sends a message to Q, without an **agreed-upon protocol**, the bits have no agreed meaning.
 - Different machines may use different **data representations** (e.g., big-endian vs. little-endian).
 - Agreement on format/protocol prevents **chaos** in communication.
 - **Keywords:** protocol, bit meaning, data representation, agreement, chaos
-

Q44. Name the 7 layers of the OSI Reference Model from bottom to top.

1. Physical
 2. Data Link
 3. Network
 4. Transport
 5. Session
 6. Presentation
 7. Application
- **Keywords:** Physical, Data Link, Network, Transport, Session, Presentation, Application
-

Q45. What are the drawbacks of the OSI reference model?

- Focuses on **message-passing only**.
 - Often has **unnneeded or unwanted functionality**.
 - **Violates access transparency**.
 - **Keywords:** message-passing, unwanted functionality, access transparency
-

Q46. What do the low-level layers (Physical, Data Link, Network) do?

- **Physical:** Specification and implementation of **bits** and their transmission.
 - **Data Link:** Transmits series of bits into **frames**; handles **error and flow control**.
 - **Network:** Describes how **packets are routed** in a network.
 - For most distributed systems, the **lowest-level interface is the network layer**.
 - **Keywords:** bits, frames, packets, routing, error control, flow control
-

Q47. What is the Transport Layer? What protocols does it provide?

- Provides **actual communication facilities** for most distributed systems.
 - **TCP:** Connection-oriented, **reliable**, stream-oriented communication.
 - **UDP:** Unreliable (**best-effort**) datagram communication.
 - Other protocols: **Real-time Transport Protocol (RTP)**, **SCTP (Stream Control Transmission Protocol)**.
 - **Keywords:** TCP, UDP, reliable, best-effort, stream, datagram, RTP, SCTP
-

Q48. What is the difference between TCP and UDP?

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable (guaranteed delivery)	Unreliable (best-effort)
Data type	Stream-oriented	Datagram
Overhead	Higher	Lower
Use case	File transfer, web	Streaming, gaming

Q49. Explain the TCP 3-Way Handshake.

- Client sends **SYN** (wants to establish connection).

- Server responds with **SYN + ACK(SYN)** (acknowledges client SYN, sends its own).
 - Client sends **ACK** (acknowledges server's SYN).
 - Three messages = **3-way handshake**; connection is now established.
 - **Keywords:** SYN, ACK, 3-way handshake, TCP, connection establishment
-

Q50. What is Client-Pull Architecture? What are its pros and cons?

- Client **sends a request** to the server to pull data.
 - Example: **HTTP** (client requests a webpage, server responds).
 - **Pros:** Server is **stateless** (no client state kept) - more robust on server crash; scalable from memory perspective.
 - **Cons:** **Less communication-efficient** - every data piece requires a request + reply (overhead).
 - **Keywords:** pull, stateless, HTTP, request-reply, robust, overhead
-

Q51. What is Server-Push Architecture? What are its pros and cons?

- Server **asynchronously pushes data** to the client without explicit client requests.
 - Example: **Video/Audio Streaming** - client sends one request, server keeps pushing data.
 - **Pros:** **Communication efficient** - less overhead (one request, continuous data).
 - **Cons:** Server is **stateful** (keeps track of clients) - **not robust** on server crash (state lost).
 - **Keywords:** push, stateful, streaming, asynchronous, not robust, efficient
-

Q52. What is the difference between Persistent and Transient Communication?

- **Persistent:** Message is **stored by middleware as long as needed** to deliver it, even if receiver is offline.
 - **Transient:** Message is stored **only while both sending and receiving applications are executing**. If middleware cannot deliver (transmission interrupt or receiver inactive), message is **discarded**.
 - **Keywords:** persistent, transient, middleware storage, discard, receiver active
-

Q53. What is the difference between Synchronous and Asynchronous Communication?

- **Synchronous:** Sender is **blocked** until request is known to be accepted.
 - **Asynchronous:** Sender **continues immediately** after submitting the message; message temporarily stored by middleware.
 - **Keywords:** blocked, continues, immediate, message store, synchronous, asynchronous
-

Q54. What is Message-Oriented Middleware (MOM)?

- Aims at **high-level persistent asynchronous communication**.
 - Processes send each other messages which are **queued**.
 - Sender **need not wait** for immediate reply.
 - Middleware ensures **fault tolerance**.
 - **Keywords:** MOM, queued, asynchronous, persistent, fault tolerance
-

Q55. What is the role of Middleware in distributed systems?

- Acts as a **software layer between OS and applications**.
 - Enables **seamless communication, coordination, and resource sharing**.
 - Provides rich set of **communication protocols, (un)marshaling** of data, **naming protocols, security protocols, scaling mechanisms** (replication, caching).
 - The main difference with OS: services offered in a **networked manner**.
 - **Keywords:** marshaling, naming, replication, caching, security, network
-

Q56. What is "Marshaling/Unmarshaling" in middleware?

- **Marshaling:** Converting data structures into a format suitable for **transmission** over a network.
 - **Unmarshaling:** Converting received data back into the original data structure format.
 - Necessary because different machines may have different **data representations**.
 - **Keywords:** marshaling, unmarshaling, data representation, transmission, middleware
-

Q57. What is MPI and mpi4py?

- **MPI (Message Passing Interface):** Standard interface for **inter-process communication** in parallel/distributed computing.
- **mpi4py:** Python library providing MPI interface.
- Key functions: `MPI.COMM_WORLD`, `comm.Get_rank()` (process ID), `comm.Get_size()` (total processes).
- Used on **clusters and supercomputers** to divide computation across processes.
- **Keywords:** MPI, mpi4py, rank, size, communicator, cluster

WEEK 4 - Parallel Architectures and Flynn's Taxonomy

Q58. What is Flynn's Taxonomy? List all four categories with examples.

Model	Instruction Streams	Data Streams	Example
SISD	1	1	Classic single-core CPU
SIMD	1	Many	GPU vector operations, image processing
MISD	Many	1	Fault-tolerant systems, special pipelines
MIMD	Many	Many	Cluster/Multicore CPU, supercomputers

- **Keywords:** SISD, SIMD, MISD, MIMD, instruction streams, data streams
-

Q59. Why is MISD considered rare in practice?

- **Multiple instruction streams** operating on the **same data stream** has few real-world applications.
 - Appears only in special **fault-tolerant schemes** and **pipelines** (e.g., space shuttle flight control systems).
 - **Keywords:** MISD, rare, fault-tolerant, pipeline, same data
-

Q60. Why is MIMD the most general and widely used model?

- Each processor **runs its own program on its own data** - fully independent.
 - Can be **synchronous or asynchronous**, deterministic or not.
 - Most **general-purpose multicore and cluster systems** fall in this category.
 - **Keywords:** MIMD, independent, synchronous, asynchronous, multicore, cluster
-

Q61. What is SIMD best suited for? Why?

- Best for **image processing, arrays, vector operations**.
 - Many processing elements run the **same instruction on different data** - **lockstep execution**.
 - **Deterministic** in behavior.
 - GPUs are a primary example of SIMD.
 - **Keywords:** SIMD, lockstep, deterministic, image, vector, GPU
-

Q62. What is UMA (Uniform Memory Access)?

- All processors access **shared memory with equal latency** - **one shared memory controller**.
 - Often called **CC-UMA** if **cache coherence** is maintained in hardware.
 - Found in **small to medium-sized desktops and servers**.
 - **Pros:** Simple to program. **Cons:** Scalability limits, contention, coherence overhead.
 - **Keywords:** UMA, equal latency, cache coherence, CC-UMA, shared memory controller
-

Q63. What is NUMA (Non-Uniform Memory Access)?

- Memory is **divided per socket**; each CPU socket has its **own local memory**.
 - Access to **local memory is faster**; accessing another socket's memory takes **more time** (due to interconnect like QPI, Infinity Fabric).
 - Found in **modern multi-socket servers** (Xeon, EPYC, Threadripper).
 - **Keywords:** NUMA, non-uniform, local memory, socket, QPI, Infinity Fabric, latency
-

Q64. What is the "First-Touch Memory Policy" in NUMA?

- Memory is allocated **physically in the bank of the CPU core that first writes to it**, not when allocated in code.
- **Correct approach:** Spawn threads - each thread initializes its own memory portion - memory becomes local to that socket - best performance.
- **Wrong approach:** Main thread initializes all memory - all memory on one socket - remote access stalls.
- **Keywords:** first-touch, NUMA-aware, thread pinning, local memory, socket

Q65. What is Thread Pinning in NUMA?

- Assigning a **thread** to a **specific CPU core or socket**.
 - If a thread runs on Socket 0 but its memory is on Socket 1 - **unnecessary latency**.
 - NUMA-aware programming = **Pin threads + allocate memory locally** = major performance boost.
 - **Keywords:** thread pinning, NUMA, socket, latency, performance
-

Q66. What are the three memory architectures in parallel computing?

Architecture	Memory Model	Scalability	Programming Ease
Shared Memory	One address space	Limited	Easy
Distributed Memory	Per-node memory	High	Harder
Hybrid	Combined	Very High	Medium

- **Keywords:** shared, distributed, hybrid, address space, scalability
-

Q67. What is Distributed Memory architecture?

- Each processor/node has its **own local memory, no global address space**.
 - To access **remote data**, tasks must explicitly **communicate** over a network.
 - **Pros:** Scales very well; local RAM access is fast; cost-effective.
 - **Cons:** Programmer must manage **communication and synchronization** explicitly.
 - **Keywords:** local memory, no global address space, explicit communication, scalable
-

Q68. What is the Shared Memory programming model?

- Multiple tasks access the **same memory address space**.
 - **Synchronization primitives** (locks, mutexes, barriers, atomics) required.
 - Problems: **Race Conditions** and **False Sharing**.
 - Used with **OpenMP, pthreads** on multi-core machines.
 - **Keywords:** shared memory, race condition, false sharing, OpenMP, pthreads, mutex
-

Q69. What is a Race Condition?

- Two threads **write to the same variable simultaneously** - **corrupted data**.
 - Must be prevented using **mutexes, locks, or atomic operations**.
 - **Keywords:** race condition, concurrent write, corruption, mutex, atomic
-

Q70. What is False Sharing?

- Threads touch **different variables** that happen to lie in the **same cache line**.
 - Causes **cache invalidations** even though logically independent data is accessed.
 - Leads to **performance degradation** on multi-core/NUMA systems.
 - **Keywords:** false sharing, cache line, invalidation, independent variables
-

Q71. What is the Message Passing (MPI) model?

- **No shared memory** - every process works in its own RAM.
 - Communication via: `MPI_Send()`, `MPI_Recv()`, `MPI_Bcast()` (broadcast), `MPI_Reduce()` (collect and combine).
 - Scales to **thousands of processes** - used by Top500 supercomputers.
 - **Keywords:** MPI, send, receive, broadcast, reduce, no shared memory, Top500
-

Q72. What is SPMD vs. MPMD?

Model	Meaning	Example
SPMD	Single Program Multiple Data - same program on all nodes, different data per rank	MPI applications
MPMD	Multiple Programs Multiple Data - different programs run cooperatively	One program generates data, another visualizes

- **Keywords:** SPMD, MPMD, rank, cooperative, MPI

Q73. What is Pipelining in parallel computing?

- Split work into **staged steps** (like an assembly line).
- **Multiple inputs flow through the stages concurrently.**
- Each stage processes one piece while the next stage works on the previous piece.
- **Keywords:** pipeline, staged, assembly line, concurrent, stages

Q74. What is the difference between Synchronous and Asynchronous execution in MPI?

- **Synchronous (MPI_Barrier):** All processes must reach the barrier before any can proceed; easier to reason about but **idle time** for faster processes.
- **Asynchronous (MPI_Isend + continue):** Process sends and continues without waiting; **faster throughput** but harder to coordinate.
- **Keywords:** MPI_Barrier, MPI_Isend, idle time, throughput, coordination

Q75. Why did parallel computing shift from MHz race to core count?

- **Power Wall:** Increasing clock frequency beyond a limit causes excessive **heat and power dissipation**.
- Solution: Use **multiple cores at lower frequency** instead of one very fast core.
- "We do not get faster by waiting for CPUs; we must design faster algorithms."
- **Keywords:** power wall, MHz race, multicore, heat, frequency

WEEK 5 - Amdahl's Law vs. Gustafson's Law

Q76. State Amdahl's Law and its formula. What does it represent?

- Gives **maximum theoretical speedup** for a **fixed workload**.
- $S(N) = 1 / [(1 - P) + (P / N)]$
 - P = parallel fraction, N = number of processors.
- Represents **upper bound of strong scaling**.
- Shows **diminishing returns** as N increases.
- **Keywords:** fixed workload, parallel fraction, strong scaling, diminishing returns

Q77. What is the maximum speedup limit in Amdahl's Law?

- As N tends to infinity: $S(\text{inf}) = 1 / (1 - P) = 1 / \alpha$
 - alpha = serial fraction.
- Even with **infinite processors**, speedup is bounded by the serial portion.
- **Keywords:** infinite processors, upper bound, serial fraction, limit

Q78. State Gustafson's Law and its formula. What does it represent?

- Assumes **problem size increases** with processor count (weak scaling).
- $S(N) = N - \alpha(N - 1)$ or equivalently $S_G(N) = \alpha + (1 - \alpha) \times N$
 - alpha = serial fraction, N = number of processors.
- More **optimistic** than Amdahl's Law.
- As N increases, speedup grows **nearly linearly**.
- **Keywords:** scaled workload, weak scaling, linear, optimistic, Gustafson

Q79. What is the difference between Strong Scaling and Weak Scaling?

Type	Law	Problem Size	Goal
Strong Scaling	Amdahl's	Fixed	Solve same problem faster
Weak Scaling	Gustafson's	Grows with N	Solve larger problem in same time

- **Keywords:** strong scaling, weak scaling, fixed, growing, Amdahl, Gustafson

Q80. Define Parallel Efficiency. What does it tell us?

- $E(N) = S(N) / N$
- Measures **utilization of processors** - what fraction of each processor's capacity is being used.
- $E = 1$ (**100%**) = perfect efficiency (ideal).
- As N increases with fixed serial fraction, **efficiency drops** due to overhead.
- If E drops sharply - **adding processors is wasteful**.
- **Keywords:** efficiency, utilization, $E = S/N$, drops, wasteful

Q81. When should you use Amdahl's Law vs. Gustafson's Law?

- **Amdahl:** Fixed workload (same job, more processors) - HPC with fixed dataset.
- **Gustafson:** Growing workload (larger job, more processors) - HPC with higher resolution, big data.
- **Key question:** "Are we speeding up the same job or solving a **bigger** job?"
- **Keywords:** fixed, growing, HPC, big data, decision

Q82. What does it mean if both Amdahl and Gustafson speedups are low?

- **Large serial bottleneck** dominates.
- Must **optimize the algorithm** - reduce serial fraction.
- Simply adding more processors will **not help**.
- **Keywords:** serial bottleneck, optimize, algorithm, not adding CPUs

Q83. What is the "marginal gain" criterion for adding processors?

- Compute $S(2N) / S(N)$.
- If doubling CPUs gives **less than 10% improvement** - **not cost-effective**.
- Adding more processors beyond this point wastes money and energy.
- **Keywords:** marginal gain, doubling, 10%, cost-effective

Q84. What is the relationship between alpha and P?

- $P = 1 - \alpha$ (parallel fraction = 1 minus serial fraction)
- $\alpha = 1 - P$ (serial fraction = 1 minus parallel fraction)
- **Keywords:** alpha, P, serial fraction, parallel fraction, complement

Q85. Interpret: "High S, Low E" in parallel performance.

- Good **absolute speed** but **diminishing returns**.
- Likely caused by: **overheads** or **load imbalance**.
- More processors are added than the work can efficiently use.
- **Keywords:** high speedup, low efficiency, overhead, load imbalance

Q86. Interpret: "High S, High E" in parallel performance.

- Algorithm **scales well**.
- **Overheads and serial fraction are small**.
- Adding more processors is still effective.
- **Keywords:** scales well, low overhead, small serial fraction

Q87. What are the real-world applications of Amdahl's Law?

1. Predicting speedup limits.
 2. Identifying **serial bottlenecks**.
 3. Guiding optimization of parallel algorithms.
- **Keywords:** speedup limit, serial bottleneck, optimization

Q88. What are the real-world applications of Gustafson's Law?

1. Designing **scalable parallel systems**.
 2. Supporting **large dataset processing**.
 3. Predicting scalability in **exascale systems**.
- **Keywords:** scalable, exascale, large datasets, Gustafson

Q89. What does it mean when Gustafson speedup $S_G(N)$ is approximately equal to N ?

- **Near-linear weak scaling** - the system scales almost perfectly.
- Every added processor contributes proportionally to the speedup.
- This is the **ideal case** for Gustafson, meaning α is approximately 0.
- **Keywords:** linear scaling, ideal, Gustafson, near-linear

Q90. What is an "Exascale System"?

- A computing system capable of **10^{18} (one exaFLOP)** operations per second.
- Current frontier in HPC; requires massive parallelism and energy efficiency.
- Gustafson's Law is particularly relevant for designing such systems.
- **Keywords:** exascale, 10^{18} , exaFLOP, HPC, Gustafson

PART 2: HIGH PROBABILITY QUESTIONS

These questions are most likely to appear **directly** in exams. Prioritize these.

HP-1. Define Distributed System with its two key characteristics.

- "A distributed system is a **collection of autonomous computing elements** that appears to its users as a single coherent system."
- **Characteristic 1 - Collection of Autonomous Nodes:**
 - Each node is independent and has its own notion of time; **no global clock** exists.
 - This leads to fundamental **synchronization and coordination problems**.
- **Characteristic 2 - Single Coherent System:**
 - Users cannot tell where computation is happening, where data is stored, or if data is replicated.
 - Nodes must **collaborate** to maintain this illusion.
 - The keyword is **distribution transparency**.
- **Keywords:** autonomous, coherent, no global clock, transparency, collaboration

HP-2. What is the difference between Amdahl's Law and Gustafson's Law?

Feature	Amdahl's Law	Gustafson's Law
Problem Size	Fixed (same job)	Scales with N (bigger job)
Scaling Type	Strong Scaling	Weak Scaling
Speedup Formula	$1 / [(1-P) + P/N]$	$N - \alpha(N-1)$
Behavior	Diminishing returns	Nearly linear growth
Optimism	Pessimistic (hard ceiling)	Optimistic
Best Used For	Fixed dataset, HPC	Growing dataset, big data

Feature	Amdahl's Law (Hard ceiling)	Gustafson's Law (Grows with N)
---------	--------------------------------	-----------------------------------

- **Amdahl:** Even infinite processors cannot exceed 1/alpha.
- **Gustafson:** Workload grows with hardware - no hard ceiling in theory.
- **Keywords:** fixed, growing, strong, weak, diminishing, linear

HP-3. Explain Flynn's Taxonomy with examples for each category.

- Flynn's Taxonomy classifies parallel architectures based on the **number of instruction streams** and the **number of data streams**.

Model	Instruction Streams	Data Streams	Example
SISD	1	1	Classic single-core CPU; sequential C loop
SIMD	1	Many	GPU vector operations; image processing
MISD	Many	1	Fault-tolerant systems; space shuttle control
MIMD	Many	Many	Multicore CPUs; cluster computing

- **SISD:** Classic serial machine - one instruction sequence operating over one data stream.
- **SIMD:** Many processing elements run the same instruction over different data simultaneously - lockstep, deterministic. Great for image/graphics and vector operations.
- **MISD:** Rare in practice - appears in special pipelines or fault-tolerant redundant systems.
- **MIMD:** Most general - each processor runs its own program on its own data independently; can be synchronous or asynchronous.
- **Keywords:** SISD, SIMD, MISD, MIMD, instruction streams, data streams, lockstep, deterministic

HP-4. What are the 7 types of Distribution Transparency?

- Distribution transparency means **hiding the fact that resources and processes are physically distributed**.

Type	What it Hides
Access	Differences in data representation and how an object is accessed
Location	Where an object is physically located
Relocation	Object moved to another location while in use (key feature of cloud computing)
Migration	Object may move to another location (general mobility of object)
Replication	Object is replicated across multiple locations
Concurrency	Object may be shared by several independent users simultaneously
Failure	Failure and recovery of an object

- **Keywords:** access, location, relocation, migration, replication, concurrency, failure, transparency

HP-5. Compare Cluster Computing and Grid Computing.

Feature	Cluster	Grid
Hardware	Homogeneous (same OS, identical hardware)	Heterogeneous (different systems)
Network	LAN (local, close-range)	WAN (dispersed, multi-organization)
Management	Single administrator / master node	Multiple organizations, virtual organizations
Scale	Smaller	Larger

Feature	Cluster	Grid
Resource Policy	Centralized control	Conflicting policies across domains
Example	Beowulf Cluster	Scientific grids

- Grid uses **virtual organizations** - groupings of users that allow authorization on resource allocation.
- **Keywords:** homogeneous, heterogeneous, LAN, WAN, virtual organization, master node

HP-6. Explain Push vs. Pull Architecture with advantages.

- **Client-Pull Architecture:**
 - Client **sends a request** to pull data from the server.
 - Example: **HTTP** - client requests a webpage, server responds.
 - Server is **stateless** - does not keep track of clients.
 - **Pros:** More robust (server crash loses no client state); scalable from memory perspective.
 - **Cons:** Less efficient - each data piece requires request + reply (2 messages = overhead).
- **Server-Push Architecture:**
 - Server **asynchronously pushes data** to the client without explicit client requests each time.
 - Example: **Video/Audio Streaming** - client sends one request, server keeps pushing data.
 - Server is **stateful** - keeps track of all clients it is communicating with.
 - **Pros:** More communication-efficient - one request, continuous data flow, less overhead.
 - **Cons:** Less robust - server crash loses all client state from memory.
- **Keywords:** pull, push, stateless, stateful, robust, efficient, HTTP, streaming

HP-7. What is TCP 3-Way Handshake? Name the 3 steps.

- TCP 3-Way Handshake is the process used to **establish a TCP connection** between client and server.
- **Step 1 - SYN:** Client sends a SYN (synchronize) packet to the server, indicating the client wants to establish a connection.
- **Step 2 - SYN + ACK:** Server sends back a SYN packet along with an ACK (acknowledgment) of the client's SYN.
- **Step 3 - ACK:** Client acknowledges the server's SYN by sending an ACK back to the server.
- After these three messages, the TCP connection is fully established and data transfer can begin.
- **Keywords:** SYN, ACK, 3-way handshake, TCP, connection establishment

HP-8. Explain UMA vs. NUMA with use cases.

- **UMA (Uniform Memory Access):**
 - All processors/cores access a single shared memory with **equal latency**.
 - One shared memory controller; any core accessing any memory location takes the same time.
 - Often called **CC-UMA** if hardware cache coherence is maintained.
 - **Use case:** Small to medium-sized desktops and servers.
 - **Pros:** Simple to program. **Cons:** Scalability limits, contention, coherence overhead.
- **NUMA (Non-Uniform Memory Access):**
 - Memory is **divided per socket** - each CPU socket has its own local memory.
 - A core accesses its **own socket's memory faster**; accessing another socket's memory takes more time due to interconnect (QPI, Infinity Fabric).
 - **Use case:** Modern multi-socket servers - Xeon, EPYC, Threadripper.
 - Requires **NUMA-aware programming**: thread pinning + first-touch memory allocation for best performance.
 - **Keywords:** UMA, NUMA, equal latency, socket, local memory, QPI, CC-UMA

HP-9. What are the 8 false assumptions in distributed system development?

- These are commonly (and incorrectly) assumed by developers, leading to fragile systems:
 1. The network is **reliable** - networks fail, packets drop.
 2. The network is **secure** - attacks, eavesdropping, and spoofing occur.
 3. The network is **homogeneous** - real networks mix many hardware/software types.
 4. The **topology does not change** - nodes join/leave, links go down.
 5. **Latency is zero** - every message takes time; WAN latency can be significant.
 6. **Bandwidth is infinite** - networks are always limited in capacity.
 7. **Transport cost is zero** - moving data costs money and resources.
 8. There is **one administrator** - real systems span multiple organizations and admin domains.
- **Keywords:** reliable, secure, homogeneous, topology, latency, bandwidth, transport cost, administrator

HP-10. What are the Memory Wall and Power Wall problems in HPC?

- **Memory Wall:**
 - **Processor speed** has grown much faster than **memory bandwidth and capacity**.

- This results in processors being **idle**, waiting for data from memory.
- DRAM grew from 16 KB (1976) to 64 GB (2011) but the gap between CPU speed and memory speed keeps widening.
- Challenge: designing memory systems that provide terabytes per second of bandwidth for data-intensive applications.
- **Power Wall:**
 - When millions of processing nodes each draw even a few watts of power, the total power dissipation reaches **megawatts**.
 - This creates severe **energy bills and cooling challenges**.
 - This is why the industry shifted from increasing clock frequency to using **multiple cores at lower frequencies**.
 - The problem applies at both supercomputer scale and personal electronics scale.
- **Keywords:** memory wall, power wall, bandwidth, DRAM, megawatts, cooling, multicore

HP-11. State Amdahl's Law and compute maximum speedup when serial fraction approaches zero.

- Amdahl's Law gives the theoretical maximum speedup for a fixed workload:
 - $S(N) = 1 / [\alpha + (1 - \alpha) / N]$
 - where α = serial fraction, N = number of processors.
- **Maximum speedup (N tends to infinity):** $S(\infty) = 1 / \alpha$
- When α approaches 0 (serial fraction approaches zero), theoretically $S(\infty)$ approaches infinity.
- However, in **real systems α can never be exactly 0** because of: communication overhead, synchronization costs, OS overhead, and load imbalance - all of which effectively increase the real serial fraction beyond what code alone suggests.
- **Keywords:** Amdahl, serial fraction, infinite processors, theoretical limit, real overhead

HP-12. Compare HPC and HTC.

Feature	HPC (High Performance Computing)	HTC (High Throughput Computing)
Goal	Solve one problem as fast as possible (low latency)	Complete many tasks per unit time (high throughput)
Metric	FLOPS measured by Linpack benchmark	Tasks per second (TPS), MIPS
Users	Scientists, engineers (less than 10% of users)	Internet and business users (majority)
Systems	Supercomputers, MPPs, clusters	Cloud, web servers, data centers
Focus	Speed of a single job	Throughput of many jobs
Example	Weather simulation, physics modeling	Google Search, web services

- **Keywords:** HPC, HTC, Linpack, FLOPS, TPS, latency, throughput

HP-13. Explain Parallel Efficiency and what it means when it drops sharply.

- **Parallel Efficiency:** $E(N) = S(N) / N$
 - Measures what fraction of each processor's capacity is actually being used productively.
 - $E = 1.0$ means perfect efficiency (ideal but unrealistic).
 - $E = 0.5$ means each processor is only 50% useful on average.
- **When efficiency drops sharply with increasing N:**
 - Over-parallelization has occurred - too many processors for the amount of parallel work available.
 - Fixed serial fraction wastes growing processor-time as N increases.
 - Adding more processors becomes **wasteful** in cost and energy.
 - This is the signal to **optimize the serial portion** of the code rather than adding more hardware.
- **Keywords:** $E = S/N$, utilization, over-parallelization, wasteful, serial fraction

HP-14. What are the three walls of HPC?

- **Memory Wall:** Memory bandwidth limits single-processor performance. The gap between processor speed and memory speed keeps growing, causing processors to wait idle for data.
- **Power Wall:** With millions of processing nodes, each drawing a few watts, total power dissipation reaches megawatts. Energy bills and cooling requirements become extreme challenges.
- **Reliability Wall:** Ensuring correct and continuous functioning of a system with thousands or millions of nodes is non-trivial. At any given time, some nodes are bound to malfunction.
- **Keywords:** memory wall, power wall, reliability wall, bandwidth, megawatts, cooling, malfunction

HP-15. What is middleware and what services does it provide?

- Middleware is the "**OS of distributed systems**" - a software layer that sits between the operating system and the application layer.
- It enables **seamless communication, coordination, and resource sharing** among distributed components.
- Services provided:
 - **Inter-application communication** via common protocols such as RPC (Remote Procedure Call).
 - **Transactions**: Atomic (all-or-nothing) execution of distributed operations.
 - **Service Composition**: Joining two or more applications (compound applications, web services).
 - **(Un)marshaling of data** for integrated systems with different data representations.
 - **Naming protocols** for easy resource sharing.
 - **Security protocols** for secure communication.
 - **Scaling mechanisms** such as replication and caching.
 - **Masking failure and recovery** from failures.
- Main difference from OS equivalents: middleware services are offered in a **networked manner**.
- **Keywords**: middleware, RPC, atomic transaction, marshaling, naming, replication, caching, security, networked

PART 3: UNEXPECTED / TRICKY QUESTIONS

These are twisted, conceptual, or combined questions designed to catch unprepared students.

TQ-1. "A distributed system with no partial failures is achievable if the middleware is robust enough." - True or False? Explain.

- **FALSE**. Partial failures are **inevitable** at any given time in a distributed system.
- No amount of middleware can **completely hide** partial failures and their recovery.
- The formal definition of a distributed system explicitly acknowledges that "hiding partial failures and their recovery is often very difficult and in general impossible to hide."
- Middleware can **mask** some failures, but it cannot prevent them from ever occurring.
- **Trap**: Students may confuse "masking failure" (a transparency type) with "eliminating failure" entirely.

TQ-2. A program with 100% parallel fraction still does not achieve infinite speedup on real hardware. Why?

- In theory, $S(\infty) = 1/\alpha = 1/0 = \text{infinity}$ when $\alpha = 0$, but this never happens in practice.
- Real systems always have hidden serial components that inflate the effective α :
 - **Communication overhead** between processors.
 - **Synchronization costs** (barriers, locks).
 - **Load imbalance** - some processors finish before others and wait idle.
 - **OS overhead** for scheduling and I/O management.
- These factors effectively **increase the real serial fraction** beyond what the code alone suggests.
- **Trap**: Confusing the theoretical Amdahl model with real-world behavior.

TQ-3. Why is Gustafson's Law more optimistic than Amdahl's Law?

- Amdahl's Law assumes the **workload is fixed** - you are speeding up the exact same job with more processors. The serial portion therefore dominates increasingly as N grows.
- Gustafson's Law assumes the **workload grows proportionally** with the number of processors, keeping all processors busy with larger and larger problems.
- In practice, many real-world tasks naturally scale - climate simulations use higher resolution, datasets grow, rendering uses more frames. This makes Gustafson more realistic in HPC.
- Gustafson does not have a hard speedup ceiling in the same way Amdahl does.
- **Trap**: Students think Gustafson is simply the same concept with a different formula.

TQ-4. What is the difference between Relocation Transparency and Migration Transparency?

- **Relocation Transparency**: Hides that an object is moved to **another location while it is actively in use**. The user does not notice any interruption. This is described as a **main feature of cloud computing** in the lecture notes.
- **Migration Transparency**: Hides that an object **may move** to another location. This refers to the general mobility of an object, not necessarily while it is being actively used.
- The key distinction: Relocation = moved while in use; Migration = moved (general mobility).
- **Trap**: These two transparency types are frequently confused or treated as identical.

TQ-5. Why did Distributed Shared Memory (DSM) fail as a concept?

- DSM tried to give multicomputers (distributed memory systems) the **ease of programming** of multiprocessors (shared memory systems).
- It used virtual-memory techniques: if process at CPU A addresses a page at CPU B, the OS at A traps and fetches the page from B.
- **Reason for failure**: Performance could **never compete** with true shared-memory multiprocessors.
- The overhead of trapping, fetching pages over the network, and managing coherence was far too high.
- It failed to meet the expectations of programmers in both performance and simplicity.
- DSM has been **widely abandoned** and is no longer used in mainstream systems.

- **Trap:** Students may think DSM is still a viable or actively used approach.
-

TQ-6. What is the difference between a "parallel program" and a "distributed program"?

- **Parallel program:** Runs on a parallel computer where processors may be tightly coupled (shared memory) or loosely coupled (distributed memory). The focus is on **speed**.
 - **Distributed program:** Runs on a distributed system consisting of fully autonomous machines with private memory communicating only via message passing over a network. The focus is on **resource sharing, fault tolerance, and geographic distribution**.
 - All distributed programs involve some degree of parallelism, but not all parallel programs run on distributed systems.
 - **Trap:** Students often treat these terms as synonyms.
-

TQ-7. If you double the number of processors and the speedup increases by only 8%, what should you do?

- This is the **marginal gain** scenario - $S(2N)/S(N) < 1.10$ means less than 10% improvement from doubling processors.
 - Adding more hardware is **not cost-effective** at this point.
 - The correct action is to **reduce the serial fraction (alpha)** of the program:
 - Optimize the sequential sections of the code.
 - Reduce synchronization barriers and lock contention.
 - Minimize I/O serial bottlenecks.
 - Reducing alpha from 5% to 2% produces far greater speedup improvement than doubling processors at this stage.
-

TQ-8. Steve Jobs said "eight cores, forget it." What fundamental computing problem was he referring to?

- Jobs was referring to the fundamental difficulty of **parallel programming**, not hardware limitations.
 - The specific problem is **TLP (Task-Level Parallelism)** - it is "far from being very successful due to difficulty in programming and compilation of code for efficient execution on multicore CMPs."
 - Writing programs that decompose correctly, avoid race conditions, manage synchronization, and achieve good load balance is extremely challenging.
 - This is also reflected in the quote: "The processor industry is adding more and more cores, but nobody knows how to program those things."
 - **Trap:** Students may only mention hardware problems or the power wall.
-

TQ-9. Can a system be both centralized and used for distributed computing? Give an example.

- **Yes.** Many data centers and supercomputers are **centralized systems** - all resources (processors, memory, storage) are in one physical location under one integrated OS.
 - However, these centralized systems are routinely used to run **parallel, distributed, and cloud computing applications**.
 - Example: A large data center may physically reside in one building (centralized) but host thousands of virtual machines serving distributed cloud applications worldwide.
 - **Trap:** Students assume centralized automatically means "not distributed in use."
-

TQ-10. Why does Amdahl's Law apply to hardware improvements too, not just parallel processors?

- Amdahl's Law is a **general principle** - it applies to any enhancement that speeds up a fraction of a task by some factor.
 - Example from the notes: Floating-point instructions improved to run 2x faster, but only 10% of actual instructions are floating-point. The overall speedup = only 1.053x because the other 90% is unaffected.
 - The unenhanced portion of the program still dominates the total execution time.
 - This means: **improving one component of a system gives diminishing returns if other components are not improved**.
 - **Trap:** Students think Amdahl applies only in the context of parallel processors.
-

TQ-11. "The network is reliable" is listed as a false assumption. Why is this dangerous if believed?

- If a developer assumes the network is always reliable, they will build **no error handling, no retransmission logic, no fault tolerance** into the system.
 - When the network inevitably fails (packet loss, disconnection, timeouts), the system will crash or produce incorrect results.
 - Real distributed systems must handle: packet loss, reordering of messages, network partitions, and intermittent failures.
 - Designing without these safeguards leads to systems that need expensive and complex patches later.
 - This is one of the 8 explicitly listed pitfalls (false assumptions) in distributed system development.
-

TQ-12. What does the "first-touch" memory policy have to do with performance in NUMA systems?

- In NUMA systems, memory is physically allocated in the bank of the CPU core that **first writes to it**, not when the memory is allocated in code.
 - **Problem scenario:** If the main thread initializes all memory (touches all pages first), all memory gets placed on the main thread's socket. When other threads running on different sockets try to access this memory, they incur **remote memory access penalties**.
 - **Solution:** Spawn all threads first, then have **each thread initialize its own portion of data**. This ensures memory is physically local to the socket running that thread.
 - The result: no remote memory access, maximum memory bandwidth, major performance boost.
 - **Trap:** Students think memory is placed based on where the malloc call is made, not on first write.
-

TQ-13. Why is the Client-Pull model "more robust from a failure perspective" but less efficient?

- **More robust:** Pull servers are **stateless** - they do not store any information about the clients they are serving. If the server crashes and restarts, it has lost nothing about its clients. The client simply resends its request.
- **Less efficient:** Every single piece of data the client wants requires **two messages**: one request from the client and one reply from the server. This 2-message overhead adds up for frequent data access.
- A push server stores all client state in memory - if it crashes, the state of all active client sessions is lost. The server must be rebuilt from scratch, causing major disruption.
- **Trap:** Students may think stateful = more reliable because it "knows more."

TQ-14. In P2P systems, which scalability problem is avoided compared to traditional distributed systems?

- P2P systems avoid the **Administrative Scalability** problem.
- Traditional distributed systems spanning multiple organizations face **conflicting policies** on usage, payment, management, and security - making cross-domain collaboration very difficult.
- P2P systems (BitTorrent, Skype, Spotify) work because **end users** collaborate directly, not administrative entities. There is no need for organizations to agree on policies.
- Size scalability and geographical scalability challenges still exist in P2P systems.
- **Trap:** Students may say P2P avoids size or geographical scalability problems.

TQ-15. If Gustafson speedup $S_G(N)$ is approximately equal to N , what does this tell you about the serial fraction?

- From Gustafson's formula: $S_G(N) = N - \alpha(N - 1)$
- If $S_G(N)$ is approximately N , then $\alpha(N - 1)$ is approximately 0.
- This means **alpha is approximately 0** (the serial fraction is negligible).
- Interpretation: Nearly all of the program is parallelizable; every added processor contributes proportionally to the total speedup; the system achieves **near-perfect linear weak scaling**.
- This is the ideal best-case scenario for a parallel system under Gustafson's model.
- **Trap:** Students may not know how to work backwards from the formula to interpret alpha.

PART 4: DIFFERENCE-BASED QUESTIONS

D-1: Parallel Computing vs. Distributed Computing

Feature	Parallel Computing	Distributed Computing
Memory	Shared OR distributed	Always private per node
Coupling	Tightly or loosely coupled	Loosely coupled (autonomous)
Communication	Shared memory OR message passing	Always message passing
Goal	Speed (faster execution)	Resource sharing, fault tolerance
Programs	Parallel programs	Distributed programs
Clock	May share clock	No global clock

D-2: HPC vs. HTC

Feature	HPC	HTC
Focus	Speed (latency)	Throughput (tasks/time)
Benchmark	Linpack (FLOPS)	TPS, MIPS
Users	Scientists, engineers	Internet, business users
System	Supercomputers, MPPs	Clusters, cloud

Feature	Weather simulation	Google Search
---------	--------------------	---------------

D-3: Cluster vs. Grid vs. Cloud

Feature	Cluster	Grid	Cloud
Homogeneity	Homogeneous	Heterogeneous	Heterogeneous
Location	LAN (close range)	WAN (multi-org)	Internet (global)
Admin	Single domain	Multiple organizations	Provider-managed
Resource mgmt	Master node	Virtual organizations	Virtualization
Scaling	Limited	Medium	Elastic

D-4: UMA vs. NUMA

Feature	UMA	NUMA
Memory access	Equal latency for all cores	Faster for local, slower for remote
Architecture	Single memory controller	Per-socket memory
Use case	Small/medium servers	Large multi-socket servers
Example	Desktop workstation	Xeon, EPYC multi-socket
Programming	Easy	Requires NUMA-aware design

D-5: Shared Memory vs. Distributed Memory

Feature	Shared Memory	Distributed Memory
Address Space	Global (one space)	Per-node (private)
Communication	Read/write variables	Explicit message passing
Scalability	Limited	High
Ease	Easy to program	Harder
Tools	OpenMP, pthreads	MPI

D-6: Synchronous vs. Asynchronous Communication

Feature	Synchronous	Asynchronous
Sender behavior	Blocked until accepted	Continues immediately
Efficiency	Lower (idle time)	Higher (no waiting)
Complexity	Simpler logic	Harder to coordinate
MPI example	MPI_Barrier	MPI_Isend

Feature Analogy	Synchronous Everyone waits at checkpoint	Asynchronous Everyone works independently

D-7: Client-Pull vs. Server-Push

Feature	Client-Pull	Server-Push
Initiator	Client requests data	Server sends without request
Server state	Stateless	Stateful
Fault tolerance	More robust	Less robust (state lost on crash)
Efficiency	Less efficient	More efficient
Scalability (memory)	More scalable	Less scalable
Example	HTTP	Video streaming

D-8: Persistent vs. Transient Communication

Feature	Persistent	Transient
Message storage	Stored until delivered	Stored only while both apps running
Receiver offline?	Message waits	Message discarded
Reliability	High	Low
Example	Email / MOM	RPC call

D-9: SPMD vs. MPMD

Feature	SPMD	MPMD
Programs	Same program on all nodes	Different programs per role
Data	Different data per rank	Different data per node
Use case	MPI parallel apps, BLAST	Producer-consumer pipelines
Complexity	Simpler	More complex

D-10: Amdahl's Law vs. Gustafson's Law (Extended)

Feature	Amdahl	Gustafson
Formula	$1/[(1-P)+P/N]$	$N - \alpha(N-1)$
Problem size	Fixed	Scales with N
Scaling type	Strong	Weak
Speedup limit	1/alpha (hard ceiling)	Grows near-linearly with N

Feature	Amdahl Pessimistic	Gustafson Optimistic
When to use	Fixed dataset, same job	Larger job, growing workload
Bottleneck	Serial fraction dominates	Serial fraction (smaller impact)

D-11: TCP vs. UDP

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable (guaranteed)	Unreliable (best-effort)
Data type	Stream	Datagram
Handshake	3-way (SYN, SYN+ACK, ACK)	None
Overhead	High	Low
Use case	File transfer, web, email	Streaming, gaming, DNS

D-12: Structured vs. Unstructured Overlay Networks

Feature	Structured	Unstructured
Neighbor set	Well-defined (tree, ring)	Randomly selected
Topology	Fixed, deterministic	Flexible, random (mesh)
Lookup	Efficient	May require flooding
Resilience	Structured failure handling	More resilient to random failures

PART 5: ONE-LINER REVISION

Last-day revision. One or two lines max. Read quickly, remember fast.

#	Concept	One-Liner
1	Distributed System	Collection of autonomous nodes appearing as a single coherent system.
2	Middleware	The "OS of distributed systems" - sits between OS and apps, provides networked services.
3	Distribution Transparency	Hiding the fact that resources are physically distributed.
4	Partial Failure	Inevitable failure of part of a distributed system; impossible to fully hide.
5	Scalability Dimensions	Size (users), Geographical (distance), Administrative (domains).
6	False Assumptions	8 false assumptions: reliable network, secure, homogeneous, zero latency, infinite bandwidth, etc.
7	Amdahl's Law	Max speedup for FIXED workload: $S = 1 / [(1-P) + P/N]$; bounded by serial fraction.

#	Concept	One-Liner
	Gustafson's Law	Speedup for GROWING workload: $S = N - \alpha(N-1)$; near-linear, optimistic.
9	Strong Scaling	Amdahl - same problem, more processors = faster.
10	Weak Scaling	Gustafson - bigger problem, more processors = same time.
11	Efficiency	$E = S/N$; drops as N grows with fixed serial fraction.
12	Amdahl Limit	$S(\infty) = 1/\alpha$ - even infinite processors cannot exceed this.
13	Flynn's Taxonomy	SISD, SIMD, MISD, MIMD - classified by instruction and data streams.
14	SIMD	Same instruction, many data - GPUs, image processing, lockstep, deterministic.
15	MIMD	Most general - each processor runs its own program on its own data.
16	MISD	Rare - multiple instructions on same data; fault-tolerant systems only.
17	UMA	All cores access memory with equal latency - simple, small-scale.
18	NUMA	Local memory is fast; remote socket memory is slow - needs NUMA-aware programming.
19	First-Touch Policy	Memory allocated on the socket that first writes to it, not when code allocates it.
20	Thread Pinning	Assigning thread to specific socket to avoid remote memory access latency.
21	Race Condition	Two threads write same variable simultaneously = corrupted data.
22	False Sharing	Threads access different variables in same cache line = unnecessary cache invalidations.
23	MPI	Message Passing Interface - no shared memory; explicit send/receive between processes.
24	SPMD	Same program, all nodes, different data per rank.
25	TCP 3-way Handshake	SYN then SYN+ACK then ACK - three messages to establish a TCP connection.
26	Client-Pull	Client requests data; server is stateless; more robust, less efficient (2 messages per request).
27	Server-Push	Server pushes data; server is stateful; less robust, more efficient (1 request, continuous data).
28	Persistent Communication	Message stored until receiver is ready - survives receiver downtime.
29	Transient Communication	Message discarded if receiver not active - only while both apps are running.
30	Asynchronous Comm	Sender continues immediately after submitting message - no blocking.
31	Synchronous Comm	Sender blocked until request is accepted by receiver.
32	HPC	High Performance Computing - measured in FLOPS (Linpack); scientists, low latency.
33	HTC	High Throughput Computing - measured in tasks/sec; Internet services, high throughput.
34	Memory Wall	Processor speed grows faster than memory bandwidth = processors wait for memory.
35	Power Wall	Increasing clock frequency creates too much heat = switched to multicore.
36	Three Walls	Memory wall + Power wall + Reliability wall = three HPC challenges.
37	Koomey's Law	Energy efficiency doubles every 1.57 years (like Moore's Law but for energy).
38	Cluster Computing	Homogeneous nodes, LAN, single admin, master node - Beowulf is an example.
39	Grid Computing	Heterogeneous, WAN, multi-org, virtual organizations - resource sharing at scale.
40	Cloud Layers	Hardware then Infrastructure then Platform then Application (IaaS, PaaS, SaaS).
41	Overlay Network	Nodes communicate only with neighbors; structured (tree, ring) or unstructured (mesh).

42 #	No Global Clock Concept	Distributed system nodes each have their own time = synchronization challenges.
43	Atomic Transaction	All-or-nothing execution of distributed operations - rollback if any step fails.
44	RPC	Remote Procedure Call - call procedure on remote machine as if local; provided by middleware.
45	Marginal Gain	If doubling CPUs gives less than 10% speedup improvement = not cost-effective to add more.
46	CPS	Cyber-Physical System - integrates Computation, Communication, and Control (3C).
47	BLP to JLP	Parallelism evolved: Bit then Instruction then Data then Task then Job level.
48	Distributed Shared Memory	Failed concept: shared memory model on multicomputers; could not match multiprocessor performance.
49	Virtual Organization	Grouping of users for authorization in grid computing; enables cross-domain collaboration.
50	Marshaling	Converting data structures for network transmission; unmarshaling = converting back.

PART 6: FORMULAS FOR NUMERICALS

SECTION A: Basic Performance Metrics

F-1: Basic Speedup

$$S = T_1 / T_N$$

Symbol	Meaning
S	Speedup
T1	Execution time on 1 processor (serial time)
TN	Execution time on N processors

When to use: You are given execution times and asked for speedup.

F-2: Parallel Efficiency

$$E(N) = S(N) / N$$

Symbol	Meaning
E(N)	Efficiency with N processors
S(N)	Speedup achieved with N processors
N	Number of processors

When to use: Compute after finding speedup; tells you how well processors are utilized. E = 1.0 means perfect efficiency; E less than 0.5 means poor utilization.

F-3: Elapsed Time

Elapsed Time = CPU Time + Wait Time

F-4: CPU Time

CPU Time = User CPU Time + System CPU Time

Elapsed Time = User CPU Time + System CPU Time + Wait Time

SECTION B: Amdahl's Law

F-5: Amdahl's Law (using Parallel Fraction P)

$$S(N) = 1 / [(1 - P) + P/N]$$

Symbol	Meaning
S(N)	Theoretical speedup with N processors
P	Parallel fraction (fraction that can run in parallel; 0 to 1)
N	Number of processors
(1-P)	Serial fraction = alpha

When to use: Fixed workload; given P and N, find speedup.

F-6: Amdahl's Law (using Serial Fraction alpha)

$$S(N) = 1 / [\alpha + (1 - \alpha)/N]$$

Symbol	Meaning
alpha	Serial fraction (alpha = 1 - P)
N	Number of processors

When to use: Same as F-5 but when given alpha (serial fraction) directly.

F-7: Amdahl's Maximum Speedup (N tends to infinity)

$$S(\text{inf}) = 1 / \alpha = 1 / (1 - P)$$

When to use: "What is the maximum possible speedup?" or "Speedup saturates at X - find serial fraction."

- To find alpha from max speedup: $\alpha = 1 / S(\text{inf})$

F-8: Execution Time After Enhancement (Amdahl original form)

$$\text{ExTimeNew} = \text{ExTimeOld} \times [(1 - \text{Fraction_enhanced}) + \text{Fraction_enhanced} / \text{Speedup_enhanced}]$$

When to use: When a specific component (like FPU) is enhanced by a factor, not the whole program parallelized.

F-9: Generalized Amdahl's Law (multiple components)

$$S = 1 / (f_1/p_1 + f_2/p_2 + \dots + f_k/p_k)$$

Symbol	Meaning
f_i	Fraction of total time spent in component i
p_i	Speedup factor for component i
$f_1 + f_2 + \dots = 1$	All fractions must sum to 1

When to use: Multiple components are enhanced by different factors simultaneously.

F-10: Find Serial Fraction from Measured Speedup (Amdahl)

Starting from: $1/S = \alpha + (1-\alpha)/N$

Rearranged step by step:

```
Multiply both sides by N:  
N/S = N*alpha + (1-alpha)  
N/S = N*alpha + 1 - alpha = 1 + alpha(N-1)  
alpha = (N/S - 1) / (N - 1)
```

When to use: Given measured speedup S and number of processors N , find the serial fraction α .

F-11: Find P from Amdahl Data

From: $1/S = (1-P) + P/N = 1 - P(1 - 1/N)$

$$P = (1 - 1/S) / (1 - 1/N)$$

When to use: Given S and N , find the parallel fraction P .

F-12: Find Minimum N for Target Speedup (Amdahl)

```
From  $S(N) \geq \text{Target}$ :  
 $1/[(1-P) + P/N] \geq \text{Target}$   
 $(1-P) + P/N \leq 1/\text{Target}$   
 $P/N \leq 1/\text{Target} - (1-P)$   
 $N \geq P / [1/\text{Target} - (1-P)]$ 
```

When to use: "Find minimum processors to achieve speedup of X ."

F-13: Parallel Execution Time from Amdahl

$$T_N = T_1 / S(N)$$

Or directly:

$$T_N = T_1 \times [\alpha + (1-\alpha)/N]$$

When to use: Given T_1 , α , N - find execution time on N processors.

F-14: Time Saved

$$\text{Time Saved} = T_1 - T_N$$

F-15: Efficiency from Amdahl (target constraint)

$$\begin{aligned} E &= S(N)/N \geq E_{\text{target}} \\ 1/[N((1-P) + P/N)] &\geq E_{\text{target}} \\ 1/[(1-P)N + P] &\geq E_{\text{target}} \\ (1-P)N + P &\leq 1/E_{\text{target}} \\ N &\leq (1/E_{\text{target}} - P) / (1-P) \end{aligned}$$

When to use: "Find maximum N that maintains efficiency $\geq E_{\text{target}}$."

SECTION C: Gustafson's Law

F-16: Gustafson's Law (Scaled Speedup)

$$S_G(N) = N - \alpha(N - 1)$$

Equivalent form:

$$S_G(N) = \alpha + (1 - \alpha) \times N$$

Symbol	Meaning
$S_G(N)$	Scaled speedup with N processors
N	Number of processors
α	Serial fraction
$(1-\alpha)$	Parallel fraction P

When to use: Workload **scales with N** (bigger problem with more processors); weak scaling scenario.

F-17: Find alpha from Gustafson Data

$$\begin{aligned} S &= N - \alpha(N-1) \\ \alpha(N-1) &= N - S \\ \alpha &= (N - S) / (N - 1) \end{aligned}$$

When to use: Given Gustafson speedup S and N, find serial fraction alpha.

SECTION D: Relationships and Checks

F-18: Relationship Between alpha and P

$$\begin{aligned} \alpha + P &= 1 \\ P &= 1 - \alpha \\ \alpha &= 1 - P \end{aligned}$$

F-19: Marginal Gain Criterion

Gain = $S(2N) / S(N)$

- If Gain < 1.10 (less than 10% improvement) - not cost-effective to double processors.

SECTION E: Quick Comparison Formulas

Scenario	Use	Formula
Basic speedup	Direct measurement	$S = T1/TN$
Efficiency	After finding S	$E = S/N$
Fixed workload speedup	Amdahl	$S = 1/[\alpha + (1-\alpha)/N]$
Max possible speedup	Amdahl limit	$S(\text{inf}) = 1/\alpha$
Growing workload speedup	Gustafson	$S = N - \alpha(N-1)$
Find alpha from Amdahl S	Reverse Amdahl	$\alpha = (N/S - 1)/(N-1)$
Find alpha from Gustafson S	Reverse Gustafson	$\alpha = (N-S)/(N-1)$
Find P from Amdahl S	Reverse Amdahl	$P = (1-1/S)/(1-1/N)$
Min N for target speedup	Amdahl rearranged	$N \geq P/[1/\text{Target} - (1-P)]$
Max N for efficiency E	Efficiency constraint	$N \leq (1/E - P)/(1-P)$
Parallel exec time	From speedup	$TN = T1/S$ or $T1 \times [\alpha + (1-\alpha)/N]$
Time saved	Basic arithmetic	$\text{Saved} = T1 - TN$
Component enhancement	Generalized Amdahl	$S = 1/(f1/p1 + \dots + fk/pk)$

SECTION F: Key Numerical Insights (Exam Tips)

- These are patterns that appear across multiple exam problems.
1. If **serial fraction alpha is small** - **system scales well** (both Amdahl and Gustafson give good speedup).
 2. If **E drops sharply with N** - **over-parallelization** - too many processors, not enough work.
 3. If **S(inf) is small** - **optimize the serial portion**, not the parallel part.
 4. Gustafson **S_G(N) approximately equal to N** - **near-perfect linear scaling** (alpha approximately 0).
 5. **Reducing alpha from 0.05 to 0.02 gives MORE improvement** than doubling processors - always reduce serial fraction first.
 6. **Mixed workloads**: Compute each part's serial contribution separately and sum:
 - $\alpha_{\text{total}} = \alpha_{\text{component1}} \times \text{fraction1} + \text{serial_IO} + \text{overhead}$
 7. **Overhead adds to serial fraction**: If overhead = 5% of total, add 0.05 to alpha before computing speedup.